

SIMATS ENGINEERING

Department of Computer Science & Engineering
ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Student Name:	V. Jeevanantham	Reg. No.:	192421234
Course Code:	CSA1205	Course Title:	Computer Architecture
CO Assessed:	CO2	Weightage:	45%
Total Marks:	25	Date:	28-07-2026

CO2 Statement: Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)

Code of Conduct: I, V. Jeevanantham (Reg. No. 192421234), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

Q1. Signed 8-bit 2's Complement Addition and Subtraction (+45 and -23)

Step 1: Binary conversion (8-bit, 2's complement)

+45 → magnitude 45 = 00101101_2 . Since it is positive, +45 = **00101101**.

23 → magnitude 23 = 00010111_2 . For -23, invert all bits (1's complement) and add 1:

$00010111 \rightarrow \text{invert} \rightarrow 11101000 \rightarrow +1 \rightarrow \mathbf{11101001} = -23$.

Step 2: Addition — (+45) + (-23) = 22

Both numbers are sign-aligned as 8-bit words before adding, and the carry is tracked bit by bit from the LSB (bit 0) to the MSB / sign bit (bit 7):

```
  00101101  (+45)
+ 11101001  (-23)
-----
  00010110   (carry out of bit 7 = 1, discarded)
```

Result = $00010110_2 = +22$ (decimal), which matches $45 - 23 = 22$.

Overflow check: the processor compares the carry INTO the sign bit (bit 7) with the carry OUT of the sign bit. Here carry-in = 1 and carry-out = 1 → they are equal, so **no overflow** occurs. This is expected because adding a positive and a negative number can never overflow an N-bit 2's-complement representation.

Step 3: Subtraction — (+45) - (-23) = 68

Subtraction $A - B$ is performed as $A + (2\text{'s complement of } B)$. Since $B = -23$, its 2's complement is +23 = 00010111 . So we compute $45 + 23$:

```
  00101101  (+45)
+ 00010111  (+23, i.e. -(-23))
-----
  01000100   (carry out of bit 7 = 0)
```

Result = $01000100_2 = +68$ (decimal), matching $45 - (-23) = 68$.

Overflow check: carry-in to sign bit = 0, carry-out of sign bit = 0 → equal, so **no overflow**. 68 is well within the representable range of an 8-bit signed number (-128 to +127), confirming the result is valid.

Interpretation

The processor detects overflow purely from the two carry bits around the sign position — it never needs to know the numbers' true decimal values. If those two carries differ, the sign bit has been corrupted by a wraparound and the result is invalid; here both operations gave matching carries, so both the addition (22) and the subtraction (68) are correctly represented in 2's complement.

Q2. 4-bit Carry Look-Ahead Adder (CLA) vs Ripple Carry Adder

Step 1: Generate and Propagate signals

For each bit position i , define:

Generate: $G_i = A_i \cdot B_i$ (a carry is generated regardless of incoming carry)

Propagate: $P_i = A_i \oplus B_i$ (an incoming carry would be passed through)

These signals let every carry be expressed directly in terms of the inputs, instead of waiting for the previous stage's carry to ripple in.

Step 2: Worked example — $A = 1001$ (9), $B = 0101$ (5)

$A_3A_2A_1A_0 = 1,0,0,1$ $B_3B_2B_1B_0 = 0,1,0,1$

$G_0 = 1 \cdot 1 = 1$, $G_1 = 0 \cdot 0 = 0$, $G_2 = 0 \cdot 1 = 0$, $G_3 = 1 \cdot 0 = 0$

$P_0 = 0$, $P_1 = 0$, $P_2 = 1$, $P_3 = 1$

Step 3: Carries computed in advance (in parallel), $C_0 = 0$

$$C_1 = G_0 + P_0 C_0 = 1$$

$$C_2 = G_1 + P_1 G_0 + P_1 P_0 C_0 = 0$$

$$C_3 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0 = 0$$

$$C_4 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0 = 0$$

Notice every C_i depends only on the original A, B inputs and C_0 — none of them waits for a neighbouring stage to finish first.

Step 4: Sum bits and final result

$S_i = P_i \oplus C_i$: $S_0=0, S_1=1, S_2=1, S_3=1 \rightarrow \text{Sum} = 1110_2 = 14$, which correctly equals $9 + 5$.

Step 5: Delay comparison and justification

In a ripple-carry adder, stage i cannot compute its sum until C_i arrives from stage $i-1$, so the worst-case delay grows linearly with the number of bits — roughly $2n$ gate delays for an n -bit adder. In the CLA, every carry is written as a two-level sum-of-products expression of the original G and P signals, so all carries are ready after a constant, fixed number of gate delays (independent of n) — only an extra XOR stage is needed afterwards to form the sums. Because the carry-generation time no longer scales with word length, CLA is significantly faster for wide (16/32/64-bit) adders, which is why it (or hybrid variants such as carry-select and carry-skip adders built from CLA blocks) is used inside modern processor ALUs instead of plain ripple-carry addition. The trade-off is more gates and wiring complexity, which is acceptable given the large speed gain.

Q3. Booth's Algorithm Multiplication: $(-6) \times (+5)$

Step 1: Setup (4-bit registers)

Multiplicand $M = -6 = 1010$ (2's complement), so $-M = +6 = 0110$.

Multiplier $Q = +5 = 0101$. Accumulator $A = 0000$. Extra bit $Q_{-1} = 0$. The algorithm runs for $n = 4$ cycles (one per multiplier bit), examining (Q_0, Q_{-1}) each time:

"00" or "11" $\rightarrow$ no arithmetic, only arithmetic-shift-right the A,Q, Q_{-1} group.

"01" $\rightarrow A = A + M$, then shift.

"10" $\rightarrow A = A - M$, then shift.

Step 2: Cycle-by-cycle trace

Step	Q0 Q-1	Operation	A	Q	Q-1
Initial	1 0	-	0000	0101	0
1	1 0	$A = A - M = 0000-1010$	0110 $\rightarrow$ shift $\rightarrow$ 0011	0010	1
2	0 1	$A = A + M = 0011+1010$	1101 $\rightarrow$ shift $\rightarrow$ 1110	1001	0
3	1 0	$A = A - M = 1110-1010$	0100 $\rightarrow$ shift $\rightarrow$ 0010	0100	1
4	0 1	$A = A + M = 0010+1010$	1100 $\rightarrow$ shift $\rightarrow$ 1110	0010	-

Each shift is an arithmetic right shift of the combined register A|Q| Q_{-1} , preserving the sign bit of A (this is what keeps negative intermediate values correctly represented).

Step 3: Final result

After 4 cycles: $A = 1110$, $Q = 0010$. The 8-bit product is A concatenated with Q = **1110 0010**. Converting from 2's complement: invert $\rightarrow$ 0001 1101, $+1 \rightarrow$ 0001 1110 = 30, so the value is **-30**, which correctly equals $(-6) \times (+5)$.

Why Booth's algorithm is efficient

Booth's algorithm handles signed numbers directly in 2's complement — no separate sign-magnitude conversion is needed. By looking at pairs of bits (Q_0, Q_{-1}) , it recognises runs of consecutive 1's in the multiplier and replaces a whole run of additions with just one subtraction at the start of the run and one

addition at the end, skipping all the additions in between. Compared to the traditional shift-and-add method, which performs one addition (or nothing) per bit unconditionally, Booth's approach reduces the number of arithmetic (add/subtract) operations, especially for multipliers with long runs of 1's or 0's, while still needing only shifts otherwise.

Q4. Division of $13 \div 3$: Restoring vs Non-Restoring

Setup

Dividend $Q = 13 = 1101$, Divisor $M = 3 = 0011$ (4-bit unsigned), Accumulator $A = 0000$, $n = 4$ iterations. In both methods, each iteration first performs an arithmetic left shift of the combined register $A|Q$ by 1 bit before acting on A .

Restoring Division

Step	Shift A Q	$A = A - M$	Decision	A (after)	Q (after)
1	0001 1010	$0001-0011=1110$ (neg)	restore: $A=A+M$, $Q_0=0$	0001	1010
2	0011 0100	$0011-0011=0000$ (≥ 0)	keep, $Q_0=1$	0000	0101
3	0000 1010	$0000-0011=1101$ (neg)	restore: $A=A+M$, $Q_0=0$	0000	1010
4	0001 0100	$0001-0011=1110$ (neg)	restore: $A=A+M$, $Q_0=0$	0001	0100

Rule: after subtracting M , if A is negative the subtraction is undone ("restored") by adding M back, and the quotient bit Q_0 is set to 0; if A stays non-negative, Q_0 is set to 1 and no restore is needed. Final result: Remainder $A = 0001 = 1$, Quotient $Q = 0100 = 4$, matching $13 = 3 \times 4 + 1$.

Non-Restoring Division

Step	Shift A Q	Operation (based on previous A sign)	Q_0	A (after)	Q (after)
1	0001 1010	$A \geq 0$ so $A=A-M=0001-0011=1110$ (neg)	0	1110	1010
2	1101 0100	A was neg so $A=A+M=1101+0011=0000$ (≥ 0)	1	0000	0101
3	0000 1010	$A \geq 0$ so $A=A-M=0000-0011=1101$ (neg)	0	1101	1010
4	1011 0100	A was neg so $A=A+M=1011+0011=1110$ (neg)	0	1110	0100

Rule: no immediate restore step. If the previous remainder was non-negative, subtract M ; if it was negative, add M instead; set $Q_0 = 1$ if the new A is non-negative, else $Q_0 = 0$. After the loop, since the final A (1110, negative) is negative, one final correction/restore is applied: $A = A + M = 1110 + 0011 = 0001$. Final result: Remainder $A = 0001 = 1$, Quotient $Q = 0100 = 4$, again matching $13 = 3 \times 4 + 1$.

Comparison

Both methods need the same 4 shift-and-subtract iterations, but restoring division performs an extra addition (the restore) in every iteration where the trial subtraction goes negative, effectively doing up to 2 arithmetic operations per step. Non-restoring division never restores mid-loop — it simply decides add-or-subtract based on the sign of the previous step and needs at most one corrective addition at the very end. This roughly halves the number of arithmetic operations in the worst case and removes the extra decision-then-undo logic, so non-restoring division is simpler to pipeline and faster in hardware, which is why it is generally preferred in modern processors.

Q5. IEEE 754 Representation and Addition (−13.25)

Step 1: Convert to normalized binary

13.25 in binary: $13 = 1101$, $0.25 = .01$, so $13.25 = 1101.01_2$. Normalizing to the form $1.xxxxx \times 2^e$: $1101.01 = 1.10101 \times 2^3$. Since the number is negative, sign bit $S = 1$.

This example illustrates the three classic IEEE 754 addition issues in practice: (1) **normalization** requires the smaller exponent's mantissa to be right-shifted before the operation, losing precision if the shift exceeds the mantissa width; (2) when signs differ, the operation becomes an effective **subtraction of magnitudes**, which can require post-normalization if leading zeros appear; and (3) **rounding** is applied whenever the true mathematical result needs more bits than the format's mantissa provides — in general this can introduce a small representation error, which is why floating-point arithmetic is not always exact for arbitrary decimal values, even though this particular example resolves without any rounding loss.